CIMALIR: Cross-Platform IoT Malware Clustering using Intermediate Representation

Minhajul Alam Rahat, Vijay Banerjee, Gedare Bloom, Yanyan Zhuang
University of Colorado Colorado Springs, CO, USA
mrahat@uccs.edu, vbanerje@uccs.edu, gbloom@uccs.edu, yzhuang@uccs.edu

***Abstract*—The widespread adoption of Internet-of-Things (IoT) devices has significantly increased the interconnectivity of everyday activities. Malware on IoT devices directly jeopardize the security and privacy of their users, and device heterogeneity poses significant challenges in malware analysis. This paper presents CIMALIR, which uses an automated approach for clustering malware that leverages an intermediate representation (IR) of low-level assembly code across different architectures. CIMALIR uses a staged analysis approach that first filters binary images using set-based similarity of function attributes before applying call graph analysis. Additionally, CIMALIR employs a string based technique to distinguish different malware families that exhibit a substantial level of shared code use. CIMALIR has a low runtime complexity as evidenced by a linear relationship between execution time and the number of functions in the binary images under analysis. Experimental results show that CIMALIR yields improved clustering results compared to the state-of-the-art binary analysis tool, BinDiff, when applied to malware binaries compiled for different instruction sets. CIMALIR results in a DBCV score of 0.75 and Silhouette Score of 0.812 when clustering malware families from the CUBE-MALIOT-2021 dataset. In comparison, BinDiff achieves scores of 0.54 and 0.616, respectively. This indicates CIMALIR's superior performance compared to BinDiff.**

***Keywords*—IoT, malware, intermediate representation**

I. INTRODUCTION

The Internet-of-Things (IoT) is prevalent in commercial, industrial, and residential systems [1], [2]. Due to a lack of strong security designs by IoT developers, IoT devices are being compromised by extant vulnerabilities such as weak authentication and malware. From 2019-2022, 20% of organizations using IoT devices have detected attacks targeting these devices [3]. Among these attacks, malware are the most common and can cause extensive damage and disruption [4]. Effective malware analysis is therefore crucial to improve IoT security.

In this paper, we tackle two major hurdles in IoT malware analysis. First, IoT devices use diverse CPU architectures, and malware exploits vulnerabilities across different device types and architectures. In 2016, the Mirai botnet primarily targeted IoT devices by launching extensive distributed denial-of-service (DDoS) attacks on platforms like Krebs on Security, OVH, and Dyn. Analysis of Mirai samples [5] identified three prevalent architectures—MIPS, ARM, and x86—across diverse IoT and embedded devices. The extant challenge of cross-platform analysis faced by malware tools [6] is even greater given IoT malware's ability to impact diverse device architectures. Current cross-platform analysis use complex structural [7]–[9], semantic [10], or behavioral [11] techniques, demanding significant computational resources. Second, a major hurdle arises when different malware families share code. Malware authors commonly practice reusing malware implementations to create new malware that can exploit new vulnerabilities. For example, Cozzi et al. [12] highlight how certain variants of Tsunami malware reuse Telnet or SSH scanners, which are also present in the Mirai and Gafgyt families for network propagation. Consequently, distinct malware families may exhibit high levels of similarity with each other, making specific malware family identification harder.

In this paper, we introduce CIMALIR, a framework for **C**ross-platform **I**oT **Mal**ware clustering using **I**ntermediate **R**epresentation. CIMALIR consists mainly of two parts: (1) cross platform analysis, and (2) code reuse analysis, yielding a resource-efficient yet accurate method for clustering malware samples across different instruction set architectures. CIMALIR is open-source and is available[1] anonymously. The main contributions of this paper are:

- We design and implement CIMALIR, a cross-architecture malware clustering framework based on IR analysis between functions that leverages set similarity of instructions, function attributes, and function call graph analysis.
- We demonstrate the application of string-based filtering in CIMALIR to cluster malware families exhibiting significant code reuse.
- We evaluate CIMALIR compared to BinDiff [13] on the CUBE-MALIOT-2021 dataset, which includes real-world IoT malware samples from various families and architectures. CIMALIR achieves a DBCV score of 0.75 and Silhouette score of 0.812, whereas BinDiff achieves 0.54 and 0.616, respectively.

II. RELATED WORK

Malware analysis can be broadly classified into dynamic and static analysis methods. Dynamic malware analysis [14] provides the ability to observe the behavior of malware even in the presence of code obfuscation, which is a major challenge for static analysis. However, executing malware samples for dynamic analysis may suffer from low code coverage, resulting in the inability to model all aspects of malware behavior. This becomes particularly challenging in the context of IoT devices where creating diverse architectural environments to safely execute malware is a daunting task. For these reasons,

[1] https://github.com/Minhaj-Rahat/Cimalir

```
0 int average(int a, int b) {
1   return (a+b)/2;
2 }
```

(a) `average` function

```
0 average
1 0010114d
2 (register, 0x0, 4) INT_ADD (register, 0x30, 4) ,
3        (register, 0x38, 4)
4 (register, 0x0, 4) INT_SDIV (register, 0x0, 4) ,
5        (const, 0x2, 4)
6 (register, 0x0, 4) COPY (register, 0x0, 4)
7  ---  RETURN (const, 0x0, 8) , (register, 0x0, 4)
```

(b) x-86 P-Code

```
0 average
1 00010420
2 (register, 0x2c, 4) INT_ADD (register, 0x20, 4) ,
3        (register, 0x24, 4)
4 (unique, 0x4100, 4) INT_SDIV (register, 0x2c, 4) ,
5        (const, 0x2, 4)
6 (register, 0x20, 4) COPY (unique, 0x4100, 4)
7  ---  RETURN (const, 0x0, 4) , (register, 0x20, 4)
```

(c) ARM P-Code

Fig. 1: Example of high P-Code decompilation.

CIMALIR uses static analysis. In the following we review the closely related work in malware static analysis.

Static analysis techniques study code or compiled features such as the control flow graph, strings, assembly instructions, and function calls. Alhanahnah et al. [15] proposed cross-architecture malware clustering using statistical signatures, Bindiff similarity scores, and N-grams of printable strings, but it struggles with string obfuscation. Nguyen et al. [16] used printable string information (PSI) for IoT botnet detection but requires manual pre-processing. Ravi et al. [9] used ELF file byte sequences with an attention-based deep learning model for IoT device malware detection, but needs diverse training data and only handles labeled data. Others have used opcode-based features [17]–[19], control flow graphs [8] and clustering techniques [20], [21]. However, these techniques are not suitable for cross-architecture analysis.

Unlike the aforementioned methods, CIMALIR takes a different approach by utilizing an assembly language IR that enables cross-architecture analysis with string analysis to distinguish malware families that have high code reuse.

III. BACKGROUND: GHIDRA AND P-CODE

CIMALIR uses Ghidra[2] as a baseline for reverse engineering and binary analysis. Ghidra uses P-Code as the intermediate representation (IR) of the low-level assembly language. P-Code is a Register Transfer Language that can specify the semantics of machine instructions. Assembly languages are different for different architectures, but P-Code models the behavior of different processors in the same way, which makes it suitable for cross-architectural analysis.

We illustrate the fundamental concept in Figure 1. Figure 1a shows a simple function `average`. Compiling the code for two different architectures, such as x86 and ARM, generates entirely different assembly instructions and memory layouts based on the architectures. CIMALIR handles such differences using an architecture-agnostic approach based on P-Code generated during Ghidra's decompilation process. The process involves two levels of abstraction: low and high P-Code [22]. In the first stage, assembly code is translated into low P-Code, which is a raw representation of processor-specific operations. Next, low P-Code is elevated to high P-Code using decompiler analysis that includes control and data flow information for a more detailed analysis of disassembled instructions. High P-Code abstracts away the low-level details of processor operations and preserves high-level operations.

Figures 1b and 1c illustrate the high P-Code operations generated for two different architectures for the C function shown in Figure 1a. The high P-Code operations (or P-Opcode) are the same for both x86 and ARM. For example, lines 2-7 in Figure 1b and Figure 1c show the same P-Opcode such as `INT_ADD`, `INT_SDIV`, `COPY`, and `RETURN`.

IV. SYSTEM OVERVIEW

The proposed framework depicted in Figure 2 is described in details below.

A. Pre-processing

We first use string analysis and entropy analysis to detect packed binaries. String analysis, using `strings`, initially identifies packed executables, with approximately 5% of binary samples showing UPX-related[3] strings. These samples are unpacked using UPX, but about 20 corrupted samples are excluded. Entropy analysis with `Detect It Easy`[4] identifies packers other than UPX. All samples from the Hajime malware family exhibited high entropy that indicates they are packed. However, the specific packer remains unidentified due to uncommon signatures. These samples are retained for clustering. After unpacking, we disassemble all the binary samples and decompile them using Ghidra. Finally, we extract high P-Code from every function in the binary for the following steps.

B. Cross Platform Similarity Analysis

1) Shortlisting Similar Functions with Jaccard Similarity (JS): Calculating the similarity among functions of binaries requires one-to-many comparisons based on function features, which can be time-consuming. Instead of using computation-intensive approaches as in [15], [23]–[25], we narrow down similar functions using Jaccard Similarity (JS) [26] of sets of opcodes used in a function. JS measures similarity by comparing the size of the intersection to the size of the union of two sets. For two sets X and Y, the time complexity of JS is $O(|X||Y|)$ [27]. Due to Ghidra's small opcode space[5], JS can efficiently find similar functions. Functions are considered similar if their JS score is above an empirical threshold to minimize false negatives. We address false positives in the next step.

2) Extracting Function Attributes: To reduce false positives resulted from JS, we further analyze function attributes from Kim et al. [28] as shown in Table I. We found that these attributes do not have an equal impact on the overall similarity of functions. Therefore, we devise the problem of finding the

[2]https://ghidra-sre.org/

[3]UPX, the Ultimate Packer for eXecutables: https://upx.github.io/

[4]https://github.com/horsicq/Detect-It-Easy

[5]https://ghidra.re/courses/languages/html/pcoderef.html

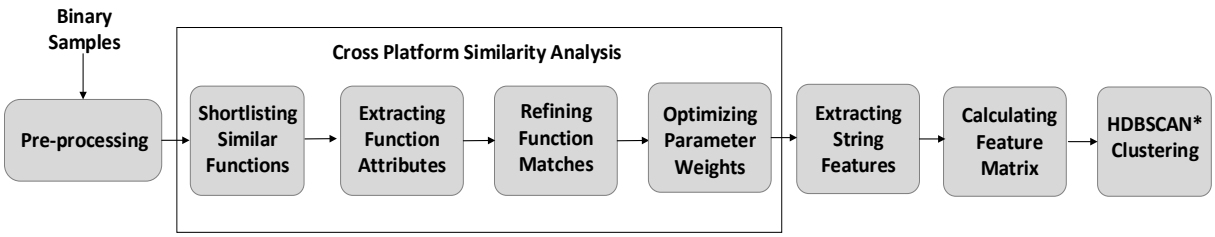

Fig. 2: Overview of CIMALIR.

TABLE I: Function attributes suggested by Kim et al. [28].

Call Graph	# of incoming calls # of outgoing calls
P-Code Instruction	# of conditional control transfer # of function parameter # of function parameters data type

optimal weights for these attributes (together with the weights introduced in Section IV-B3) as an optimization problem in Section IV-B4. The optimization approach assigns a higher weight to the attributes that have a more significant impact on the similarity of functions. From the weighted values, we generate a feature vector for each function and calculate the weighted Euclidean distance among the vectors to measure function similarity. With well-selected features, dissimilar functions have greater feature distance than similar ones. Finally, we keep the functions with the Euclidean distance below an empirically determined distance for further processing and discard the remaining.

3) Refining Function Matches using Call Graph: In some cases, two functions may match based on JS but differ in their functionality. We call such matches ambiguous. Ambiguous matches can also occur where multiple dissimilar functions have a small Euclidean distance due to inaccurate disassembly [29]. To address ambiguous matches, we use function call graphs to capture the functionality and context in which these functions are employed within the code. We first find the callers and callees for the matched functions in the call graph. For each function, we extract its P-Code instructions along with those of its callers and callees, and we combine them to make a call graph P-Code set. We then compare the call graph P-Code sets using JS for function matching. If the JS score for two functions' call graph P-Code sets exceeds an empirical threshold, they are considered similar.

4) Optimizing Parameter Weights: The final decision of similarity between binaries depends on a set of parameters: JS threshold (Section IV-B1), function attribute weights in Table I, their Euclidean distance threshold (Section IV-B2), and function call graph JS threshold (Section IV-B3). Our goal is to find the combination of values for these parameters to maximize the similarity score between samples from the same malware family and minimize for different malware families. We model this as an optimization problem with the following objective function:

$$\max \sum_{1}^{n} (\text{Similarity}_{ab} - \text{Similarity}_{ac}) \tag{1}$$

where a and b are samples from the same family, c is from a different family than a and b, and n is the number of times $(\text{Similarity}_{ab} - \text{Similarity}_{ac})$ is calculated for randomly selected samples $\{a, b, c\}$ from two randomly selected families. In our case, $n = 30$. The objective function aims to prevent misclassification of samples that belong to different families as the same family. The combination of parameters for the objective function in Equation (1) has a large search space, so we use a genetic algorithm to solve this problem. We evaluate effectiveness of this approach in Section V-B1.

C. Extracting String Features

Code reuse among malware families can lead to the challenge of high false positives in clustering due to substantial function matches, as mentioned in Section I. To tackle this challenge, we use unique printable strings to identify malware samples from different families despite code reuse. For example, malware samples in Tsunami create an IRC botnet by using specific IRC commands that can be identified as strings in the binary. If an IoT device is compromised by this malware, the device sends a `NICK` command to an IRC server to change the nickname of the compromised client[6]. The server replies with a `PING` command to the compromised device, which in turn responds with a `PONG`. These specific commands can group Tsunami samples together instead of incorrectly classifying them as samples in another family like Gafgyt, from which Tsunami reuses the same code.

D. Calculating Feature Matrix

CIMALIR uses two types of features for clustering malware: similarity scores among binaries and features extracted from malware strings. We describe each in the following.

1) Similarity Score and Matrix: After matching similar functions between binaries, the similarity score S is calculated as the ratio between the matched functions (F_m) and the maximum number of functions (F_{max}) present in either of the binaries, i.e., $S = \frac{F_m}{F_{max}}$. Calculating the similarity score using the largest number of functions in the denominator prevents the situation when a binary containing fewer functions could appear similar to a larger binary due to code reuse. We calculate each sample's similarity score with every other sample in the dataset, resulting in an $N \times N$ similarity matrix, where N is the total number of samples.

[6]https://github.com/shipcod3/IRC-Bot-Hunters/blob/master/malicious_samples/kaiten.c

TABLE II: Sample Distribution among Architectures.

Malware Family	ARM	MIPS
Gafgyt	148	81
Dofloo	216	0
Tsunami	71	41
Mirai	187	311
MrBlack	4	0
Hajime	91	100
Total	1,250	

2) String Features: Once n unique strings are identified from all the malware samples (described in Section IV-C and $n = 11$ in our case), we incorporate these strings as features in the existing feature vectors of size N. This results in a feature matrix of dimension $(N+n) \times (N+n)$ for the entire dataset. This feature matrix is used to cluster the malware samples into different families in the next step.

E. Clustering

In this step, we use Hierarchical Density-based Spatial Clustering of Applicants with Noise (HDBSCAN*) [30], which is an unsupervised clustering algorithm that does not require training or a labeled dataset. It detects clusters using mutual reachability distance as the distance metric [31]. HDBSCAN* automatically determines the number of clusters, is well-suited for imbalanced data, and reduces misclassifications by treating samples that do not fit in a cluster as noise. We select HDBSCAN* parameters (minimum cluster size and minimum samples) using brute force search to maximize the DBCV score defined in Equation (2). With the optimal parameters from Section IV-B4, HDBSCAN* groups malware samples into their respective clusters.

V. Experiments

In this section, we empirically evaluate CIMALIR based on the relative impact of features, the performance of clustering, and its runtime performance. We also compare CIMALIR with the state-of-the-art binary analysis tool BinDiff.

A. Dataset

We use malware samples in the CUBE-MALIOT-2021 dataset[7] to evaluate the efficacy of CIMALIR. This dataset has not been previously used for cross-platform malware clustering analysis. The dataset contains ELF binaries compiled for ARM (32 bit) and MIPS (32- and 64-bit) platforms, encompassing stripped and non-stripped versions, as well as packed and unpacked binaries. Additionally, the dataset includes both statically and dynamically linked binaries.

We use 1,250 malware samples from this dataset, which contains samples from the following malware families: Gafgyt, Mirai, Tsunami, Hajime, Dofloo and MrBlack. The families are labeled using the Virus Total scanners[8]. These labels are then used as the ground truth for evaluation in Section V-B2. However, we notice that for a given sample, not every scanner gives the same result. Therefore, during the labeling process, we take the majority vote for the labels in case of differences between scanners. The distribution of the malware samples among different architectures is shown in Table II.

[7]https://github.com/CrySyS/cube-maliot-2021

[8]https://www.virustotal.com/gui/home/upload

TABLE III: Weights and Distance Thresholds.

Parameters	Value
Jaccard Score	0.7
Caller Weight	87
Callee Weight	74
Parameter Weight	73
Control Transfer Weight	22
Distance Threshold	10
Call Graph Jaccard Similarity Threshold	0.85

B. Evaluation

In this section, we describe parameter weight selection and evaluate the clustering performance of CIMALIR.

1) Selection of Parameter Weights: For the optimization problem formulated in Section IV-B4, we use a genetic algorithm with 100 generations. Each generation consists of a population size of 50, and we use tournament parent selection, simulated binary crossover, and polynomial best mutation. Parameter optimization is performed on a subset of 20% of the binary samples in the dataset. We call this subset the *validation* dataset. The parameter values found by the genetic algorithm are shown in Table III.

2) Clustering Performance: To assess CIMALIR's effectiveness, we compare its clustering performance with BinDiff [13], a state-of-the-art binary analysis tool. In our analysis, we use the remaining 80% of the dataset consisting of 1,000 binary samples with the parameter weights in Table III.

BinDiff does not support unpacking. For a fair comparison, we use the preprocessed, unpacked binaries as described in Section IV-A to compare the performance of CIMALIR with BinDiff. We use IDA Pro to export disassembly data into the Protocol Buffer format that BinDiff requires. To evaluate the clustering performance, we generate two matrices of feature vectors. One feature matrix is generated with CIMALIR, while the other is generated using BinDiff. BinDiff excludes any string-based comparison when analyzing binaries. As a result, we decide to omit the string features when using BinDiff and instead rely on the similarity scores generated by BinDiff as the primary feature.

We use the following seven internal and external cluster validation metrics [32] to assess the quality of clustering.

DBCV score is a relative validity index that evaluates the quality of clustering [33]. A validity index determines the density of a cluster in relation to the density between clusters. A higher density within clusters and a lower density between clusters result in a higher validity index, and thus better clustering quality. The DBCV score is the weighted average of the validity indexes of all clusters:

$$DBCV(K) = \sum_{i=1}^{K} \frac{|K_i|}{N} V_K(K_i), \tag{2}$$

where $V_K(K_i)$ denotes the validity index of the ith cluster K_i, K is the number of clusters, and N is the total number of samples.

Homogeneity evaluates the distribution of malware families F with regard to a cluster assignment C. In a perfect clustering, a cluster c groups together samples from exactly one malware family f, resulting in a homogeneity score of 1 [34]. Homogeneity is defined as $h = 1 - \frac{H(F|C)}{H(F)}$, where

$H(F|C)$ is the conditional entropy of malware samples from families F given the cluster assignment C, and is given by

$$H(F|C) = -\sum_{f=1}^{|F|}\sum_{c=1}^{|C|}\frac{n_{f,c}}{N}.\log\left(\frac{n_{f,c}}{n_c}\right), \quad (3)$$

where N is the total number of samples, n_f and n_c are the number of samples that belong to family f, and assigned to cluster c respectively, and $n_{f,c}$ is the number of samples *both* from family f and assigned to cluster c [35]. $H(F)$ is the entropy of families F and is given by

$$H(F) = -\sum_{f=1}^{|F|}\frac{n_f}{N}.\log(\frac{n_f}{N}). \quad (4)$$

Completeness evaluates the distribution of a cluster assignment C with regard to malware families F. In a perfect clustering, all samples from the same family f will be placed in exactly one cluster c, resulting in a completeness score of 1 [34]. Completeness is defined as $c = 1 - \frac{H(C|F)}{H(C)}$, where the conditional entropy of cluster assignment C given families F, $H(C|F)$, and the entropy of cluster assignment C, $H(C)$, are defined in a symmetric way as $H(F|C)$ and $H(F)$ above.

V-measure is the weighted harmonic mean of homogeneity and completeness [34], and a higher V-Measure shows a better clustering solution. V-measure is defined as $v = \frac{(1+\beta)\times h\times c}{(\beta\times h)+c}$, where β controls the weights of homogeneity and completeness scores, where a value of 1 for this parameter indicates the same weight for the two scores.

Adjusted mutual information [36] measures the agreement between two assignments U and V. In our case, assignment U is a malware sample's true family label, and V is the family label assigned by the clustering algorithm. The higher the value, the better the clustering is compared to the ground truth. Adjusted mutual information is defined as

$$AMI(U,V) = \frac{MI(U,V) - E(MI(U,V))}{\frac{H(U)+H(V)}{2} - E(MI(U,V))} \quad (5)$$

where $E()$ denotes an expected value, $MI(U,V)$ is the mutual information between U and V, and $H(U)$ and $H(V)$ denote the entropy of U and V, respectively.

Calinski-Harabasz index measures the ratio of inter-cluster and intra-cluster variance. For a good clustering algorithm, the resulting inter-cluster variance will be high and intra-cluster variance will be low, and will lead to a higher value for this index. The Calinski-Harabasz index is defined as

$$CH = \frac{trace(S_B)}{trace(S_W)}.\frac{N-1}{N-K} \quad (6)$$

where S_B is the between-cluster dispersion matrix, S_W is the within-cluster dispersion matrix, N is the total number of malware samples, and K is the number of clusters determined by the clustering algorithm [32]. The dispersion matrices calculate the variance of the samples.

Silhouette score compares the separation of inter-cluster and intra-cluster samples determined by the clustering algorithm [37]. Silhouette score for a single sample in a dataset is computed as $s = \frac{b-a}{max(a,b)}$, where a is the arithmetic mean of the distance between the sample and all other samples within its assigned cluster, and b is between this sample and all other samples in the nearest cluster. The value of the score ranges from -1 to 1. The closer the value is to 1, the better the cluster separation is.

TABLE IV: Clustering Performance.

Metric	Score	
	CIMALIR	BinDiff
DBCV Score	0.75	0.54
Homogeneity	0.985	0.848
Completeness	0.664	0.448
V-Measure	0.793	0.586
Adjusted Mutual Information	0.790	0.578
Calinski-Harabasz Index	5706.536	925.144
Silhouette Score	0.812	0.616

TABLE V: Cluster Distribution among Malware Families

Malware Family	Number of Clusters		Number of Noise Samples	
	CIMALIR	BinDiff	CIMALIR	BinDiff
Gafgyt	1	5	3	15
Tsunami	1	4	0	19
Mirai	5	10	4	41
Dofloo	2	2	2	0
MrBlack	1	1	4	0
Hajime	2	3	0	6

C. Results

Table IV presents the clustering performance for CIMALIR and BinDiff. CIMALIR consistently outperforms BinDiff in all of the clustering measures. CIMALIR demonstrates a 38.9% improvement in the DBCV score, indicating its superior capability to partition clusters and generate more compact partitions compared to BinDiff. CIMALIR's homogeneity score is 0.985 whereas BinDiff achieves only 0.848. This indicates that clusters resulting from CIMALIR are more homogeneous than BinDiff and therefore better at separating the malware families into different clusters. Enhanced scores for completeness and v-measure are also evident in CIMALIR.

The completeness score is lower than the homogeneity score in both CIMALIR and BinDiff. This is because for malware families like Mirai, Hajime, and Dofloo, the samples are placed in more than one cluster, as shown in Table V, due to the presence of multiple variants in the same malware family. There can be deviation among these variants in terms of how they exploit vulnerabilities in different devices, and even within the same variant, there may be differences depending on the specific devices they target for exploitation [38]. As a result, when comparing these variants using a pairwise function matching, the similarity scores tend to be low. This has resulted in their placement in different clusters.

Table V additionally highlights the samples categorized as noise (outliers) by both CIMALIR and BinDiff. Notably, clusters derived from CIMALIR contain fewer noise samples than Bindiff. Furthermore, improved adjusted mutual information, Calinski-Harabasz index, and Silhouette score indicate better separation among clusters using CIMALIR.

1) Impact of Function Call Graph: We perform clustering experiments both with and without the use of call graph to assess its impact on clustering performance. The clusters are generated using the method outlined in Section V-B2 excluding the integration of string features. Without call graph,

TABLE VI: Impact of Call Graph

Malware Family	Number of Clusters	
	No Call Graph	With Call Graph
Gafgyt	20	5
Tsunami	10	3
Mirai	28	6
Dofloo	3	2
MrBlack	1	1
Hajime	4	2

TABLE VII: Impact of String Feature

Metric	CIMALIR		BinDiff	
	No Strings	With Strings	No Strings	With Strings
DBCV Score	0.568	0.75	0.54	0.675
Homogeneity	0.872	0.985	0.848	0.946
Completeness	0.658	0.664	0.448	0.70
V-Measure	0.793	0.793	0.586	0.804
Adjusted Mutual Information	0.652	0.790	0.578	0.802
Calinski-Harabasz Index	3139.621	5706.536	925.144	3556.896
Silhouette Score	0.794	0.812	0.616	0.698

we observe a significant amount of noise, with 105 samples considered noise compared to 31 when using the call graph. By incorporating call graph, we can refine ambiguous function matches and improve the similarity among samples from the same malware family. This, in turn, leads to a reduction in the number of clusters in each malware family, particularly for the Gafgyt, Tsunami, and Mirai malware families. Table VI shows the considerable improvement in clustering performance through the introduction of call graph.

2) Impact of String Features: To assess the impact, we perform clustering by excluding the string features, and then subsequently compare the results with those obtained when considering the string features. The comparison is presented in the Table VII. From the table, it is apparent that there is noticeable improvement across all the metrics when using string features. Table VII also illustrates that introducing string feature to BinDiff improves its clustering performance.

3) Execution Time: Figure 3 shows CIMALIR's runtime performance to generate the similarity score relative to the total number of functions being compared. CIMALIR's execution time is proportional to the number of functions being compared (Pearson's Correlation coefficient $\rho = 0.996$), indicating a positive linear correlation. Overall, CIMALIR takes 2.4s to calculate the similarity score for an average of 438,290 function comparisons. Thus, CIMALIR achieves better clustering performance without substantial overhead, which is crucial in malware analysis. Figure 4 illustrates that CIMALIR's use of Jaccard Similarity results in a much lower execution time compared to using only call graph. By excluding Jaccard Similarity and using only call graph, the average execution time increases from 2.4s to 99.7s to calculate the similarity score for an average of 721,760 instructions.

D. Discussion

In this section, the clustering performance across different architectures demonstrates CIMALIR's efficacy. However, there are some inherent limitations. CIMALIR may not function optimally in case of function inlining, which can alter the P-Code in a function. However, CIMALIR can overcome this challenge by implementing selective function inlining techniques proposed by Chandramohan et al. [39]. Furthermore,

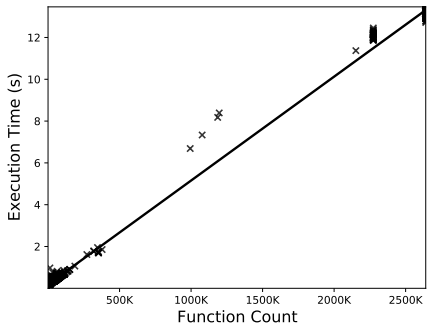

Fig. 3: Execution Time.

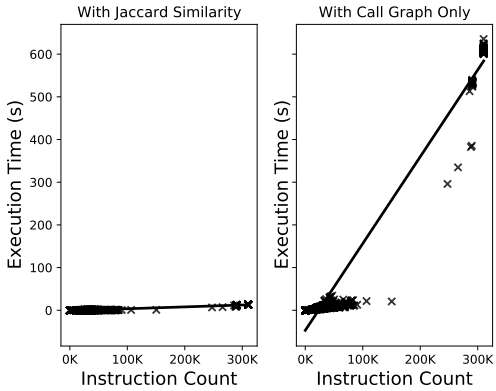

Fig. 4: Impact of Jaccard Similarity on Execution Time.

in the presence of obfuscation [40], [41], CIMALIR may encounter challenges in achieving the optimal performance.

VI. Conclusion and Future Work

We introduce CIMALIR as an automated malware clustering framework based on IR extracted from malware samples on different architectures. We also introduce string-based similarity analysis for separating malware families with high code reuse. The evaluation shows that CIMALIR can cluster malware samples into different families more effectively than BinDiff. To tackle the challenges of evasion tactics, future work may improve performance by incorporating dynamic analysis and explore the effects of malware obfuscation and evasion [42].

References

[1] P. Agbaje, A. Anjum, A. Mitra, E. Oseghale, G. Bloom, and H. Olufowobi, "Survey of interoperability challenges in the internet of vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 12, pp. 22838–22861, 2022.

[2] G. Bloom, B. Alsulami, E. Nwafor, and I. C. Bertolotti, "Design patterns for the industrial internet of things," in *2018 14th IEEE International Workshop on Factory Communication Systems (WFCS)*, pp. 1–10, 2018.

[3] G. reserach, "Iot security primer: Challenges and emerging practices." https://www.gartner.com/en/doc/iot-security-primer-challenges-and-emerging-practices, 2022 [Online].

[4] L. E. S. Jaramillo, "Malware detection and mitigation techniques: lessons learned from mirai ddos attack," *Journal of Information Systems Engineering & Management*, vol. 3, no. 3, p. 19, 2018.

[5] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, *et al.*, "Understanding the mirai botnet," in *26th {USENIX} security symposium ({USENIX} Security 17)*, pp. 1093–1110, 2017.

[6] A. Duby, T. Taylor, G. Bloom, and Y. Zhuang, "Evaluating feature robustness for windows malware family classification," in *2022 International Conference on Computer Communications and Networks (ICCCN)*, pp. 1–10, 2022.

[7] H. Alasmary, A. Anwar, J. Park, J. Choi, D. Nyang, and A. Mohaisen, "Graph-based comparison of iot and android malware," in *International Conference on Computational Social Networks*, pp. 259–272, Springer, 2018.

[8] T. N. Phu, L. H. Hoang, N. N. Toan, N. D. Tho, and N. N. Binh, "Cfd-vex: A novel feature extraction method for detecting cross-architecture iot malware," in *Proceedings of the Tenth International Symposium on Information and Communication Technology*, pp. 248–254, 2019.

[9] V. Ravi, T. D. Pham, and M. Alazab, "Attention-based multidimensional deep learning approach for cross-architecture iomt malware detection and classification in healthcare cyber-physical systems," *IEEE Transactions on Computational Social Systems*, 2022.

[10] D. Vasan, M. Alazab, S. Venkatraman, J. Akram, and Z. Qin, "Mthael: Cross-architecture iot malware detection based on neural network advanced ensemble learning," *IEEE Transactions on Computers*, vol. 69, no. 11, pp. 1654–1667, 2020.

[11] C.-I. Fan, H.-W. Hsiao, C.-H. Chou, and Y.-F. Tseng, "Malware detection systems based on api log data mining," in *2015 IEEE 39th annual computer software and applications conference*, vol. 3, pp. 255–260, IEEE, 2015.

[12] E. Cozzi, P.-A. Vervier, M. Dell'Amico, Y. Shen, L. Bilge, and D. Balzarotti, "The tangled genealogy of iot malware," in *Annual Computer Security Applications Conference*, pp. 1–16, 2020.

[13] "Bindiff manual." https://zynamics.com/bindiff/manual/, 2022 [Online].

[14] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, "Dynamic malware analysis in the modern era—a state of the art survey," *ACM Computing Surveys (CSUR)*, vol. 52, no. 5, pp. 1–48, 2019.

[15] M. Alhanahnah, Q. Lin, Q. Yan, N. Zhang, and Z. Chen, "Efficient signature generation for classifying cross-architecture iot malware," in *2018 IEEE conference on communications and network security (CNS)*, pp. 1–9, IEEE, 2018.

[16] H.-T. Nguyen, Q.-D. Ngo, and V.-H. Le, "A novel graph-based approach for iot botnet detection," *International Journal of Information Security*, vol. 19, no. 5, pp. 567–577, 2020.

[17] H. HaddadPajouh, A. Dehghantanha, R. Khayami, and K.-K. R. Choo, "A deep recurrent neural network based approach for internet of things malware threat hunting," *Future Generation Computer Systems*, vol. 85, pp. 88–96, 2018.

[18] C.-W. Tien, S.-W. Chen, T. Ban, and S.-Y. Kuo, "Machine learning framework to analyze iot malware using elf and opcode features," *Digital Threats: Research and Practice*, vol. 1, no. 1, pp. 1–19, 2020.

[19] H. Darabian, A. Dehghantanha, S. Hashemi, S. Homayoun, and K.-K. R. Choo, "An opcode-based technique for polymorphic internet of things malware detection," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 6, p. e5173, 2020.

[20] G. Pitolli, L. Aniello, G. Laurenza, L. Querzoni, and R. Baldoni, "Malware family identification with birch clustering," in *2017 International Carnahan Conference on Security Technology (ICCST)*, pp. 1–6, IEEE, 2017.

[21] H. Faridi, S. Srinivasagopalan, and R. Verma, "Performance evaluation of features and clustering algorithms for malware," in *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*, pp. 13–22, IEEE, 2018.

[22] N. Naus, F. Verbeek, D. Walker, and B. Ravindran, "A formal semantics for p-code," in *Verified Software. Theories, Tools and Experiments.: 14th International Conference, VSTTE 2022, Trento, Italy, October 17–18, 2022, Revised Selected Papers*, (Berlin, Heidelberg), p. 111–128, Springer-Verlag, 2023.

[23] M. Hassen and P. K. Chan, "Scalable function call graph-based malware classification," in *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*, pp. 239–248, 2017.

[24] Y. Ki, E. Kim, and H. K. Kim, "A novel approach to detect malware based on api call sequence analysis," *International Journal of Distributed Sensor Networks*, vol. 11, no. 6, p. 659101, 2015.

[25] G. Liang, J. Pang, and C. Dai, "A behavior-based malware variant classification technique," *International Journal of Information and Education Technology*, vol. 6, no. 4, p. 291, 2016.

[26] S. Niwattanakul, J. Singthongchai, E. Naenudorn, and S. Wanapu, "Using of jaccard coefficient for keywords similarity," in *Proceedings of the international multiconference of engineers and computer scientists*, vol. 1, pp. 380–384, 2013.

[27] M. D. Singh, P. R. Krishna, and A. Saxena, "A privacy preserving jaccard similarity function for mining encrypted data," in *TENCON 2009-2009 IEEE Region 10 Conference*, pp. 1–4, IEEE, 2009.

[28] D. Kim, E. Kim, S. K. Cha, S. Son, and Y. Kim, "Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned," *IEEE Transactions on Software Engineering*, 2022.

[29] C. Pang, R. Yu, Y. Chen, E. Koskinen, G. Portokalidis, B. Mao, and J. Xu, "Sok: All you ever wanted to know about x86/x64 binary disassembly but were afraid to ask," in *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 833–851, IEEE, 2021.

[30] R. J. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Advances in Knowledge Discovery and Data Mining: 17th Pacific-Asia Conference, PAKDD 2013, Gold Coast, Australia, April 14-17, 2013, Proceedings, Part II 17*, pp. 160–172, Springer, 2013.

[31] G. Stewart and M. Al-Khassaweneh, "An implementation of the hdbscan* clustering algorithm," *Applied Sciences*, vol. 12, no. 5, p. 2405, 2022.

[32] E. Rendón, I. Abundez, A. Arizmendi, and E. M. Quiroz, "Internal versus external cluster validation indexes," *International Journal of computers and communications*, vol. 5, no. 1, pp. 27–34, 2011.

[33] D. Moulavi, P. A. Jaskowiak, R. J. Campello, A. Zimek, and J. Sander, "Density-based clustering validation," in *Proceedings of the 2014 SIAM international conference on data mining*, pp. 839–847, SIAM, 2014.

[34] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure," in *Proceedings of the 2007 joint conference on empirical methods in natural language processing and computational natural language learning (EMNLP-CoNLL)*, pp. 410–420, 2007.

[35] "Homogeneity, completeness and v-measure." https://scikit-learn.org/stable/modules/clustering.html#homogeneity-completeness, 2023 [Online].

[36] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: is a correction for chance necessary?," in *Proceedings of the 26th annual international conference on machine learning*, pp. 1073–1080, 2009.

[37] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *Journal of computational and applied mathematics*, vol. 20, pp. 53–65, 1987.

[38] M. Dib, S. Torabi, E. Bou-Harb, N. Bouguila, and C. Assi, "Evoliot: A self-supervised contrastive learning framework for detecting and characterizing evolving iot malware variants," in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pp. 452–466, 2022.

[39] M. Chandramohan, Y. Xue, Z. Xu, Y. Liu, C. Y. Cho, and H. B. K. Tan, "Bingo: Cross-architecture cross-os binary search," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pp. 678–689, 2016.

[40] T. Blazytko, M. Contag, C. Aschermann, and T. Holz, "Syntia: Synthesizing the semantics of obfuscated code.," in *USENIX Security Symposium*, pp. 643–659, 2017.

[41] A. Mohanta, A. Saldanha, A. Mohanta, and A. Saldanha, "Malware packers," *Malware Analysis and Detection Engineering: A Comprehensive Approach to Detect and Analyze Modern Malware*, pp. 189–211, 2020.

[42] A. Duby, T. Taylor, G. Bloom, and Y. Zhuang, "Detecting and Classifying Self-Deleting Windows Malware Using Prefetch Files," in *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 0745–0751, Jan. 2022.